

SYSTEM DOCUMENTATION

InstaPrint: Self-Service Kiosk Printing System

Complete Project Context, Architectural Specification, and
Technical Reference Manual

Author: InstaPrint Technologies

Version: 2.0.0 (Production-Ready)

Date: August 12, 2026

Platform: NodeJS / Express / Windows spooler

Table of Contents

1. Executive Overview	3
2. System Architecture Specifications	3
3. Repository Structure & Directory Map	4
4. Core Module Analyses	5
5. System Security Protocols & Guards	7
6. Database Schema & Storage Entities	8
7. Core Logic & Control Flows	10
8. API Endpoint Reference Directory	11
9. Printer Cooldown Safety Algorithm	13
10. Testing & Compilation Guidelines	13

1. Executive Overview

InstaPrint is a self-service printing kiosk system tailored for retail operations. In traditional setups, customers must transfer document files directly to shopkeepers via email, chat, or external USB drives. This process creates bottleneck delays, compromises user privacy, and exposes systems to malware threats.

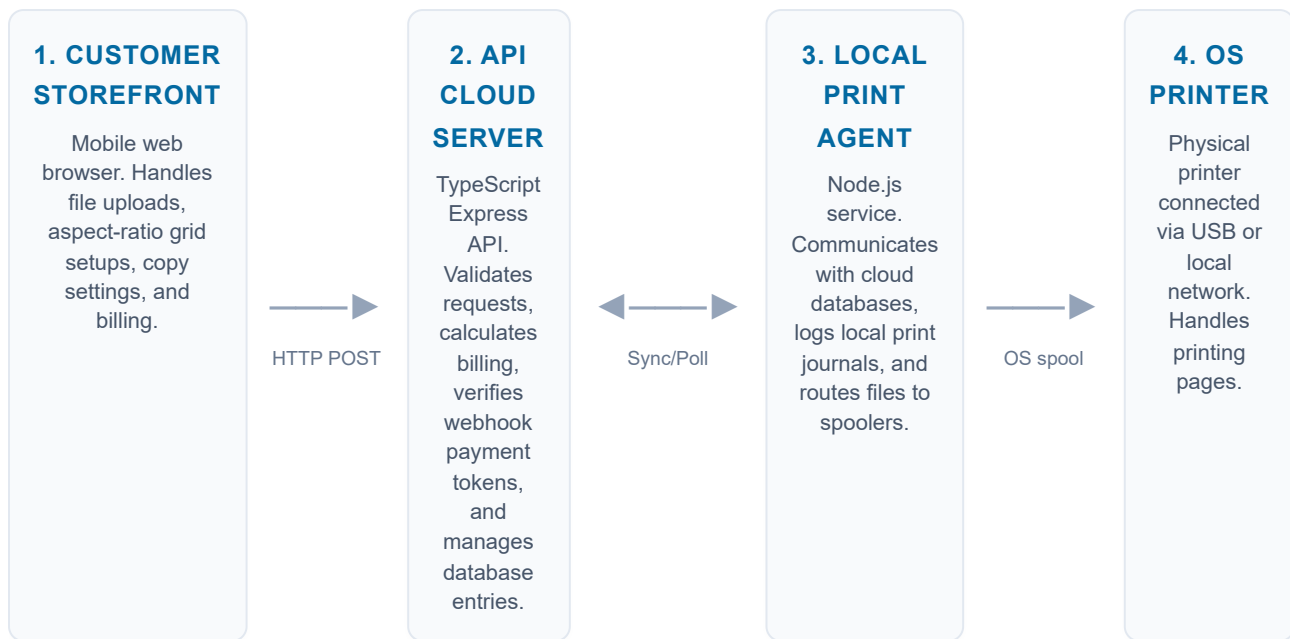
InstaPrint resolves these issues by dividing operations into three autonomous modules. The customer scans a QR code generated at the counter to open a local storefront. They select files, select page setups, format photo layouts, pay via standard gateways, and receive a pickup token. The local agent running on the PC polls the cloud server, fetches files, checks hardware variables, and prints pages automatically.

Key Core Value Propositions

- **Autonomous Queue Management:** FIFO queues sort active transactions, allowing automated operations without merchant oversight.
- **Privacy Protection:** Document uploads are verified client-side, encrypted during transit, and purged from backend directories after 30 minutes.
- **Physical Hardware Safeguards:** Embedded safety limits monitor page limits and pause active queues for printer cooldowns when thresholds are reached.

2. System Architecture Specifications

InstaPrint operates as a cloud-coordinated local print queue. The customer storefront connects to the backend to upload documents and authorize billing. The local print agent listens to the backend to download files and spool them locally.



Communication between modules is designed for network resilience:

- **Front-to-Back:** Web browsers upload documents via REST APIs. Payments are routed through Razorpay SDKs and validated on the backend.
- **Back-to-Agent:** The local agent polls REST endpoints or subscribes to Firebase Realtime references. Transactions use JSON structures.
- **Agent-to-OS:** Dispatches jobs using PDF payloads compiled with `pdf-lib` and spooled via native system drivers.

3. Repository Structure & Directory Map

The codebase is structured logically to separate the cloud backend, local hardware agent, and consumer storefront.

```
print vending/ |— InstaPrint-Agent-Distribution/ # Packaged distribution binaries | |— .env #
Local distribution configuration | |— InstaPrint-Agent.exe # Compiled Node binary executable via
pkg |— agent/ # Local Print Agent Source Code | |— agent.js # Main agent service & Local Express
dashboard | |— config.json # Persisted printer mapping and operator configurations | |—
dashboard.html # Shopkeeper Console dashboard web UI | |— jobs_journal.json # Local journal buffer
for offline recovery | |— package.json # Dependencies (pdf-to-printer, pdf-lib, firebase-admin) |—
backend/ # Cloud API Server Source Code | |— server.ts # Main entrypoint, routes, and security
validators | |— stores.ts # Abstracted data access layers for Firebase / Supabase | |—
supabase_schema.sql # SQL schema definitions, RLS policies, and RPC triggers | |— server.test.ts #
Integration Jest test suite for API endpoints | |— package.json # Dependencies (express, helmet,
razorpay, nodemailer, zod) |— frontend/ # Customer Web Storefront Source Code | |— index.html #
Customer mobile storefront viewport | |— app.js # Core client logic, page counters, Razorpay UI
integration | |— index.css # Styling (glassmorphism UI, HSL tokens) | |— sw.js # Service Worker
for offline asset caching | |— manifest.json # PWA configuration |— start_kiosk.js # Root
automation startup and configuration synchronizer |— package.json # Root script orchestrator and
concurrently execution configurations
```

4. Core Module Analyses

4.1 Root Orchestrator (start_kiosk.js)

To simplify deployment, `start_kiosk.js` manages network tunnels and environment configurations automatically.

On startup, it spawns Cloudflare Tunnels using the packaged `cloudflared.exe`. If named tunnels are not configured, it generates quick tunnels for the storefront (port 8080) and API server (port 5002). It reads the Cloudflare process logs to parse the dynamic public domains:

```
const match = content.match(/https:\/\/[a-z0-9-]+\\.trycloudflare\.com/i);
```

Once URLs are verified, the script rewrites `API_BASE_URL` within `frontend/app.js` and updates `.env` configurations for both the backend and local agent. This ensures storefronts and agents communicate correctly across dynamic network environments.

4.2 Customer Storefront (frontend/)

The storefront provides a modern mobile-optimized printing interface.

- **Client-Side Page Counting:** When a user uploads a PDF, the client loads `pdf-lib` in the browser to parse the document structure:

```
const pdfDoc = await PDFLib.PDFDocument.load(bytes);  
const pageCount = pdfDoc.getPageCount();
```

This allows the client to estimate costs immediately before files are uploaded. Text files estimate page counts assuming 55 lines per page.

- **Smart Grid Packing:** For printing multiple images (e.g., passport photos), the storefront packs layouts onto virtual pages (presets 1, 2, 4, 6, 8, or custom). It rearranges elements in the preview container to minimize paper waste.
- **Checkout Processing:** Integrates with Razorpay for secure card and UPI payments. If in *Demo Mode*, a mock transaction is sent to the backend, enabling developers to simulate successful checkouts without hitting gateway charges.

4.3 Secure Cloud API Server (backend/)

The backend is written in TypeScript and uses Express. It acts as the gateway for files and transaction records.

- **Security Enhancements:** Secures header payloads using `helmet` and configures precise CORS rules that only accept requests from verified storefront domains.
- **Access Control:** Integrated rate-limiters block endpoint abuse:

Limiter	Window	Max Requests	Action / Error Response
Global Rate Limiter	15 min	100	Blocks excessive traffic from matching IPs.
Login Limiter	15 min	5	Blocks brute-force login attempts.
Upload Limiter	1 min	5	Limits document uploads.
Checkout Client Limiter	1 min	3	Limits checkout submissions.
Checkout Shop Limiter	1 min	5	Limits checkout load per shop (grouped by shop ID).

- **Storage Abstractions:** Exposes standardized interfaces (`ISettingsStore` , `IJobStore`) implemented across Firebase, Supabase, or in-memory arrays. This allows the backend to adapt to different cloud configurations without rewriting core logic.

4.4 Local Print Agent (agent/)

The print agent runs locally on the operator's PC and handles physical print spooling.

It communicates with the database to monitor pending jobs. When a new job is received, it downloads the document, verifies the file integrity, and routes it to local printer drivers using `pdf-to-printer` commands:

```
const ptp = require('pdf-to-printer');
await ptp.print(filePath, { printer: printerName });
```

It serves the local Shopkeeper Console on port 3000, allowing operators to map printer destinations, adjust pricing, configure batch limits, and monitor queue progress.

5. System Security Protocols & Guards

InstaPrint features multi-layered security gates to protect system resources and hardware:

5.1 Magic-Byte Binary Inspection

Attackers may try to bypass extension filters to upload malicious executables. To block this, the backend inspects the magic byte headers of uploaded base64 files:

```
function isValidMagicBytes(buffer: Buffer): boolean {
  if (buffer.length < 4) return false;

  // PDF Magic Bytes: %PDF-
  if (buffer[0] === 0x25 && buffer[1] === 0x50 && buffer[2] === 0x44 && buffer[3] === 0x46) return true

  // PNG Magic Bytes: \x89PNG
  if (buffer[0] === 0x89 && buffer[1] === 0x50 && buffer[2] === 0x4E && buffer[3] === 0x47) return true

  // JPEG Magic Bytes: \xFF\xD8\xFF
  if (buffer[0] === 0xFF && buffer[1] === 0xD8 && buffer[2] === 0xFF) return true;

  // WEBP Magic Bytes: "RIFF" at offset 0 and "WEBP" at offset 8
  return buffer.length >= 12 &&
    buffer[0] === 0x52 && buffer[1] === 0x49 && buffer[2] === 0x46 && buffer[3] === 0x46 &&
    buffer[8] === 0x57 && buffer[9] === 0x45 && buffer[10] === 0x42 && buffer[11] === 0x50;
}
```

Uploaded files that do not match these signatures are rejected instantly.

5.2 Payment Verification & Webhook Protection

Payments are verified by calculating a SHA256 HMAC signature using the raw request body and the configured secret key. The computed signature is compared with the gateway's header signature using `crypto.timingSafeEqual` to prevent timing-based analysis:

```
const hmac = crypto.createHmac('sha256', secret);
hmac.update(rawBody);
const expectedSignature = hmac.digest('hex');

const sigBuf = Buffer.from(signature, 'utf8');
const genSigBuf = Buffer.from(expectedSignature, 'utf8');
const isValid = sigBuf.length === genSigBuf.length &&
  crypto.timingSafeEqual(sigBuf, genSigBuf);
```


5.3 Google reCAPTCHA v3 Verification

The checkout and payment endpoints verify reCAPTCHA tokens generated on the storefront. Requests with a verification score below 0.5 are flagged as automated bot traffic and blocked:

```
const verifyUrl = `https://www.google.com/recaptcha/api/siteverify?secret=${secretKey}&response=${token}`;
const response = await axios.post(verifyUrl);
if (response.data.success && response.data.score >= 0.5) { next(); } else { /* Blocked */ }
```

6. Database Schema & Storage Entities

The database layer supports both Firebase Realtime Database and Supabase PostgreSQL backends.

6.1 Supabase Schema Definitions

The PostgreSQL database runs on Supabase. It uses Row Level Security (RLS) policies to isolate shop profiles, settings, print queues, and token counters.

1. Table: `public.profiles`

Stores shop credentials and operator information.

```
CREATE TABLE IF NOT EXISTS public.profiles (  
  shop_id TEXT PRIMARY KEY,  
  owner_id UUID REFERENCES auth.users(id) ON DELETE SET NULL,  
  email TEXT DEFAULT '',  
  shop_name TEXT NOT NULL,  
  upi_id TEXT DEFAULT '',  
  password_hash TEXT DEFAULT '',  
  qr_code TEXT DEFAULT '',  
  qr_code_url TEXT DEFAULT '',  
  created_at TIMESTAMPTZ DEFAULT timezone('utc'::text, now()) NOT NULL  
);
```

2. Table: `public.settings`

Stores pricing and active printer mappings.

```
CREATE TABLE IF NOT EXISTS public.settings (  
  shop_id TEXT PRIMARY KEY REFERENCES public.profiles(shop_id) ON DELETE CASCADE,  
  bw_price NUMERIC DEFAULT NULL,  
  color_price NUMERIC DEFAULT NULL,  
  max_pages_per_batch INTEGER DEFAULT 80 NOT NULL,  
  cooldown_min INTEGER DEFAULT 5 NOT NULL,  
  printers JSONB DEFAULT '{}'::jsonb NOT NULL,  
  updated_at TIMESTAMPTZ DEFAULT timezone('utc'::text, now()) NOT NULL  
);
```

3. Table: `public.print_queue`

Manages customer print jobs.

```
CREATE TABLE IF NOT EXISTS public.print_queue (  
  id TEXT PRIMARY KEY,  
  shop_id TEXT NOT NULL REFERENCES public.profiles(shop_id) ON DELETE CASCADE,  
  file_url TEXT NOT NULL,  
  print_type TEXT CHECK (print_type IN ('bw', 'color')) NOT NULL,  
  total_pages INTEGER NOT NULL,  
  copies INTEGER NOT NULL,  
  token_number TEXT NOT NULL,  
  status TEXT CHECK (status IN ('pending', 'pending_payment', 'printing', 'completed', 'failed')) DEFAU  
  paid BOOLEAN DEFAULT false NOT NULL,  
  cost NUMERIC NOT NULL,  
  payment_id TEXT DEFAULT '' NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT timezone('utc'::text, now()) NOT NULL,  
  printed_at TIMESTAMPTZ,  
  priority BOOLEAN DEFAULT false NOT NULL,  
  paper_size TEXT DEFAULT 'A4' NOT NULL,  
  error_message TEXT DEFAULT ''  
);
```

4. Table: `public.daily_token_counters`

Tracks daily printed token sequences atomically.

```
CREATE TABLE IF NOT EXISTS public.daily_token_counters (
  shop_id TEXT REFERENCES public.profiles(shop_id) ON DELETE CASCADE,
  day_str TEXT NOT NULL,
  counter INTEGER DEFAULT 0 NOT NULL,
  PRIMARY KEY (shop_id, day_str)
);
```

6.2 Row Level Security (RLS) Access Rules

The tables enforce RLS policies to restrict read/write permissions based on the user's role:

Target Table	Anonymous (Public) Access	Authenticated Owner Access
<code>profiles</code>	Select (Read profile)	Insert / Update / Delete (Only matching `owner_id`)
<code>settings</code>	Select (Read prices)	Insert / Update / Delete (Only matching shop owner)
<code>print_queue</code>	Insert / Select (Submit & status check)	Update / Delete (Only matching shop owner)
<code>daily_token_counters</code>	Insert / Select / Update	Insert / Select / Update

6.3 Atomic Token Trigger Function

To prevent race conditions when generating customer tokens (e.g., two checkout requests getting the same token), the system runs an atomic increment database function:

```
CREATE OR REPLACE FUNCTION public.increment_daily_token(p_shop_id text, p_day_str text)
RETURNS integer AS $$
DECLARE
  v_counter integer;
BEGIN
  INSERT INTO public.daily_token_counters (shop_id, day_str, counter)
  VALUES (p_shop_id, p_day_str, 1)
  ON CONFLICT (shop_id, day_str)
  DO UPDATE SET counter = daily_token_counters.counter + 1
  RETURNING counter INTO v_counter;
  RETURN v_counter;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

7. Core Logic & Control Flows

7.1 Local Agent Print Journal & Offline Durability

The local print agent implements an offline journal to ensure job statuses are preserved during network outages.

When the agent receives a job, it writes the state to `jobs_journal.json` :

```
{
  "job_id_xyz": {
    "id": "job_id_xyz",
    "jobData": { "totalPages": 5, "copies": 1, "tokenNumber": "12-1" },
    "localStatus": "printing",
    "remoteStatus": "pending",
    "pendingSync": true,
    "updatedAt": 1723485600000
  }
}
```

If the network disconnects during printing, the agent proceeds offline. The local status changes are buffered in the journal. Once the connection is re-established, the sync worker reviews the journal, uploads the updated statuses, and clears the `pendingSync` flags.

7.2 ETA Estimation Algorithm

The storefront calculates and displays a dynamic estimated wait time. It queries active jobs for the shop and computes the ETA using the following formula:

```
const basePrepSeconds = 30;
const printSecondsPerPage = 5;
const totalSeconds = basePrepSeconds + (pendingPagesCount + (totalPages * copies)) * printSecondsPerPage;
const etaMinutes = Math.ceil(totalSeconds / 60);
```

7.3 Completed Queue Sorting Mechanics

To help operators monitor active queues, the Shopkeeper Console dynamically sorts the job list:

- **Pending & Printing:** Kept at the top of the queue dashboard to highlight active jobs.
- **Failed:** Displayed with error logs directly below active jobs for immediate troubleshooting.
- **Completed:** Auto-sorted to the bottom of the list to clear the workspace.

8. API Endpoint Reference Directory

The backend exposes endpoints grouped under the `/api/v1/` API route. Legacy routes are retained for backward compatibility.

8.1 Public Storefront Endpoints

1. GET `/api/v1/settings/:shopId`

Retrieves pricing configurations and site keys.

Response:

```
{
  "bwPrice": 2.00,
  "colorPrice": 10.00,
  "maxPagesPerBatch": 80,
  "cooldownMin": 5,
  "upiId": "merchant@bank",
  "recaptchaSiteKey": "6L..."
}
```

2. POST `/api/v1/payments`

Creates a Razorpay order. If gateway credentials are not configured, it generates a development mock order.

Request Body:

```
{
  "printType": "color",
  "totalPages": 10,
  "copies": 2,
  "clientId": "client_abc",
  "shopId": "default_shop"
}
```

3. POST `/api/checkout`

Validates payment signatures, generates a sequential token, queues the print job, and dispatches customer notifications.

Request Body:

```
{
  "fileUrl": "http://server/uploads/file.pdf",
  "printType": "color",
  "totalPages": 10,
  "copies": 2,
  "clientId": "client_abc",
  "customerContact": "customer@email.com",
  "shopId": "default_shop",
  "status": "pending",
  "payment": {
    "orderId": "order_xyz",
    "paymentId": "pay_123",
    "signature": "hmac_sig_here"
  }
}
```

4. GET `/api/v1/pickup/:token`

Allows customers to poll print status updates.

Response:

```
{
  "token": "12-1",
  "status": "completed",
  "totalPages": 10,
  "copies": 2,
  "cost": 200,
  "eta": "0 min"
}
```

8.2 Local Print Agent & Dashboard Endpoints

1. GET `/api/jobs`

Used by the local agent to fetch active queues.

Response:

```
{
  "job_1": {
    "id": "job_1",
    "fileUrl": "http://...",
    "printType": "bw",
    "totalPages": 5,
    "copies": 1,
    "tokenNumber": "12-1",
    "status": "pending",
    "cost": 10.00
  }
}
```

2. POST `/api/jobs/:id/status`

Used by the print agent to report progress and printing states.

Request Body:

```
{
  "status": "completed",
  "cost": 10.00,
  "printedAt": 1723485650000,
  "paid": true
}
```

8.3 Operator Configuration Endpoints (Secured)

1. POST `/api/v1/settings/:shopId`

Updates shop settings. Requires a valid authorization header token.

Request Header: `Authorization: Bearer [JWT_TOKEN]`

Request Body:

```
{
  "bwPrice": 1.50,
  "colorPrice": 8.00,
  "maxPagesPerBatch": 100,
  "cooldownMin": 4,
  "upiId": "new_merchant@bank"
}
```


2. POST `/api/v1/printers/:shopId`

Registers a new hardware printer mapping.

Request Body:

```
{
  "name": "HP LaserJet 400",
  "colorMode": "bw",
  "maxPages": 80,
  "cooldownMin": 5,
  "paperSize": "A4",
  "scale": "fit"
}
```

9. Printer Cooldown Safety Algorithm

Physical printers can overheat or experience mechanical errors when printing large batches continuously. The local agent mitigates this with an automated cooldown tracking loop:

Hardware Cooldown Safety Protocol

When the cumulative printed page count exceeds the configured batch limit, the agent pauses processing, updates the dashboard status, and initiates a safety cooldown timeout.

- **Continuous Count Tracker:** Tracks the number of printed pages.
- **Threshold Check:** When a new print job is fetched, the agent checks if `cumulativePageCount + jobPages > maxPagesPerBatch` (default: 80 pages).
- **Cooldown Phase:** If the threshold is exceeded, the agent:
 1. Pauses print queue polling.
 2. Sets the console status to **COOLDOWN ACTIVE**.
 3. Schedules a recovery timeout matching the shop settings (`cooldownMin` - default: 5 minutes).
- **Recovery Phase:** Once the timer expires, the counter is reset to 0, and the agent resumes active queue polling.

10. Testing & Compilation Guidelines

10.1 Running Backend Tests

The backend integration tests use Jest and Supertest. The test suite validates schema checking, file validation, signature verification, rate limits, and endpoint routers.

To run the test suite, execute the following command in the backend directory:

```
cd backend
npm run test
```

10.2 Database Migrations

To migrate data from a legacy Firebase Realtime Database setup to a Supabase PostgreSQL instance, use the migration utility:

```
node backend/migrate_to_supabase.js
```

Ensure the destination database tables are initialized by applying the `supabase_schema.sql` script before running migrations.

10.3 Compiling the Agent Executable

The local agent can be packaged into a standalone executable (`InstaPrint-Agent.exe`) for operators. This packages the Node runtime and dashboard templates:

```
cd agent
npm run build:exe
```

This runs `pkg` configuration assets defined in `agent/package.json` :

```
"pkg": {
  "assets": ["dashboard.html"],
  "scripts": ["agent.js"],
  "targets": ["node18-win-x64"],
  "outputPath": "dist"
}
```